实验二　线性变换与直方图变换

一、实验目的与要求

1. 了解图像增强的目的及意义，巩固所学理论知识。
2. 掌握直接灰度变换的图像增强方法。
3. 掌握灰度直方图的概念及其计算方法。
4. 熟练掌握直方图均衡化的计算过程。
5. 学会分析图像直方图。

二、实验原理及知识点

1、线性变换

灰度线性变换是一种灰度变换，通过建立灰度映射来调整源图像的灰度，达到图像增强的目的。灰度映射通常使用灰度变换曲线来表示。灰度线性变换就是将图像的像素值通过指定的线性函数进行变换，以此增强或减弱图像的灰度。灰度线性变换的公式是常见的一维线性函数：

$$g(x,y) = k \cdot f(x,y) + b$$

设 x 为原始灰度值，则变换后的灰度值 y 为：

$$y = k \cdot x + b \quad (0 \le y \le 255)$$

k 表示直线的斜率，b 表示线性函数在 y 轴的截距。

k b取值	意义
$k > 1$	增大图像的对比度，图像的像素值在变换后全部增大，整体效果被增强
$k = 1$	通过调整b，实现对图像亮度的调整
$0 < k < 1$	图像的对比度被削弱
$k < 0$	原来图像亮的区域变暗，原来图像暗的区域变亮

下面给出线性变换增强图像的 MATLAB 和 Python 函数:

Matlab: output = imadjust(input,[low_in high_in],[low_out high_out])

Python: output = linear_transform (input, low_in, high_in, low_out, high_out)

2、直方图变换

直方图均衡化是一种简单有效的图像增强技术，通过改变图像的直方图来改变图像中各

像素的灰度，主要用于增强动态范围偏小的图像的对比度。原始图像由于其灰度分布可能集中在较窄的区间，造成图像不够清晰。例如，过曝光图像的灰度级集中在高亮度范围内，而曝光不足将使图像灰度级集中在低亮度范围内。采用直方图均衡化，可以把原始图像的直方图变换为均匀分布（均衡）的形式，这样就增加了像素之间灰度值差别的动态范围，从而达到增强图像整体对比度的效果。换言之，直方图均衡化的基本原理是：对在图像中像素个数多的灰度值（即对画面起主要作用的灰度值）进行展宽，而对像素个数少的灰度值（即对画面不起主要作用的灰度值）进行归并，从而增大对比度，使图像清晰，达到增强的目的。

下面给出直方图均衡化增强图像对比度的 MATLAB 和 Python 函数:

Matlab: output = histeq(input)

Python: output = cv2.equalizeHist(input)

3、实验完整代码

Matlab 完整代码：

```
I=imread('testimg/camera.png');
figure(1),subplot(2,2,1),imshow(I),title('原始灰度图像')
figure(2),subplot(2,2,1),imhist(I),title('灰度图像的直方图')

% 用 histeq 函数进行直方图均衡化
H=histeq(I);
figure(1),subplot(2,2,2),imshow(H),title('直方图均衡化后图像')
figure(2),subplot(2,2,2),imhist(H),title('均衡化后图像的直方图')

% 用 imadjust 函数进行线性变换
I1=imadjust(I,[0 1],[1 0]);
figure(1),subplot(2,2,3),imshow(I1),title('线性变换 1 后图像')
figure(2),subplot(2,2,3),imhist(I1),title('线性变换 1 后的直方图')

I2=imadjust(I,[0.3 0.7],[0.1 0.9]);
figure(1),subplot(2,2,4),imshow(I2),title('线性变换 2 后图像')
figure(2),subplot(2,2,4),imhist(I2),title('线性变换 2 后的直方图')
```

Python 完整代码：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
def linear_transform(img, low_in=0, high_in=1, low_out=0, high_out=1):
    assert high_in >= 0 and high_in <= 1 and low_in >= 0 and low_in <= 1
    assert high_out >= 0 and high_out <= 1 and low_out >= 0 and low_out <= 1
    img = img / 255.0
    out = np.zeros_like(img)
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            if img[i,j] < low_in:
                out[i,j] = low_out
            elif img[i,j] > high_in:
                out[i,j] = high_out
            else:
                k = (high_out-low_out)/(high_in-low_in)
                b = (high_in*low_out-low_in*high_out)/(high_in-low_in)
                out[i,j] = img[i,j] * k + b
    out = out * 255.0
    return k,b,out

def main():
    grayimg = cv2.imread('testimg/camera.png', cv2.IMREAD_GRAYSCALE)
    plt.figure(figsize=(8, 8))                                  #  显示原始图像
    plt.subplot(2, 2, 1), plt.imshow(grayimg, cmap='gray'), plt.title('original img')
    H = cv2.equalizeHist(grayimg)                               #  直方图均衡化并显示图像
    plt.subplot(2, 2, 2), plt.imshow(H, cmap='gray'), plt.title('after histeq')
    # cv2.imwrite('H.png', H)
    _, _, I1 = linear_transform(grayimg, 0, 1, 1, 0)            #  线性变换 1 并显示图像
    plt.subplot(2, 2, 3), plt.imshow(I1, cmap='gray'), plt.title('after linear transform1')
    _, _, I2 = linear_transform(grayimg, 0.3, 0.7, 0.1, 0.9)    #  线性变换 2 并显示图像
    plt.subplot(2, 2, 4), plt.imshow(I2, cmap='gray'), plt.title('after linear transform2')
    plt.figure(num=2, figsize=(8, 8))                           #  显示各图像直方图
    plt.subplot(2, 2, 1), plt.hist(grayimg.flatten(), bins=256), plt.title('original hist')
    plt.subplot(2, 2, 2), plt.hist(H.flatten(), bins=256), plt.title('after histeq')
    plt.subplot(2, 2, 3), plt.hist(I1.flatten(), bins=256), plt.title('after linear transform1')
    plt.subplot(2, 2, 4), plt.hist(I2.flatten(), bins=256), plt.title('after linear transform2')
```

```
    plt.show()
    # 另一种画直方图的方法
    # his_ori = cv2.calcHist([grayimg], [0], None, [256], [0, 255])
    # plt.subplot(2, 2, 1), plt.plot(his_ori), plt.title('original hist')
if __name__ == '__main__':
    main()
```

三、实验内容及步骤

Matlab 部分：

1．利用 imread()函数读取一幅灰度图像；

2．利用 imshow()和 imhist()命令显示原始灰度图像及其直方图；

3．利用 histeq()函数对原始图像进行直方图均衡化处理；

4．利用 imshow()和 imhist()命令显示直方图均衡化后的图像及其直方图；

5．利用 imadjust()函数对原始图像进行两次线性变换处理。第一次线性变换将[0-1]灰度映射到[1-0]灰度，第二次线性变换将[0.3-0.7]灰度映射到[0.1-0.9]灰度；

6．利用 imshow()和 imhist()命令显示两次线性变换后的图像及其直方图；

7．观察不同结果的图像和直方图质量及特征；

8．将每一步的函数执行语句拷贝下来，写入实验报告，并且将得到第 3、5 步得到的图像效果保存。保存直方图显示结果（可以截图）。

Python 部分：

1．利用 cv2.imread()函数读取一幅灰度图像，并用 plt.imshow()函数显示；

2．利用 cv2.equalizeHist()函数将原始灰度图像进行直方图均衡化处理，并显示；

3．调用 linear_transform ()函数对原始灰度图像进行两次线性变换处理；第一次线性变换将[0-1]灰度映射到[1-0]灰度，第二次线性变换将[0.3-0.7]灰度映射到[0.1-0.9]灰度，显示两张线性变换后的结果图像；

4．利用 plt.hist()函数，显示上述所有图像(包括原始图像和三张处理后图像)的直方图；

5．观察不同结果的图像和直方图质量及特征；

6．将每一步的函数执行语句拷贝下来，写入实验报告，并且将得到第 2、3 步得到的图像效果保存。保存直方图显示结果（可以截图）。

四、考核要点

1、 熟悉在 Matlab / Python 中如何对图像进行直方图均衡化和线性变换、如何显示图像的直方图信息，熟悉相关的处理函数。

2、 明确不同的图像文件格式，由于其具体的图像存储方式不同，所以文件的大小不同，因此当对同一幅图像来说，有相同的文件大小时，质量不同。

五、实验仪器与软件

(1) PC 计算机

(2) MatLab 软件/语言包括图像处理工具箱(Image Processing Toolbox) ;
Python 语言环境/编辑器/图像处理 OpenCV 扩展包(cv2)/ matplotlib 扩展包

(3) 实验所需要的测试图片

六、实验报告要求

描述实验的基本步骤，用数据和图片给出各个步骤中取得的实验结果和源代码，并进行必要的讨论，必须包括原始图像及其计算/处理后的图像。

七、思考题

(1) Matlab 中，img2=imadjust(img1, [X1 X2], [Y1 Y2])函数的具体公式是什么？

(2) 图像直方图均衡化的原理，及其在图像增强中的优劣点

(3) 怎么对 RGB 图像做直方图均衡化？*可能存在的问题、如何解决？

八、实验图像

Fig.4 camera.png